

03_transfer_learning_comparison

July 5, 2026

1 DATA PREPARATION

```
[1]: import os
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.applications import MobileNetV2, EfficientNetB0
from tensorflow.keras.models import Model
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
```

1.0.1 1. Define Directory Paths

```
[2]: !unzip -q banana_dataset.zip -d dataset/
```

```
[3]: base_dir = '/content/dataset'
train_dir = os.path.join(base_dir, 'train')
val_dir = os.path.join(base_dir, 'valid')
```

1.0.2 Setting Image Parameters

```
[4]: IMG_HEIGHT = 416
IMG_WIDTH = 416
BATCH_SIZE = 16 # Reduced to 16 to prevent Colab GPU Out-of-Memory errors
```

1.0.3 Configure Data Generators (Augmentation for training, none for validation)

```
[5]: train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)
val_datagen = ImageDataGenerator(rescale=1./255)
```

1.0.4 Loading the Data

```
[6]: print("Loading Training Data...")
train_data = train_datagen.flow_from_directory(
    train_dir, target_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE,
    class_mode='categorical')

print("\nLoading Validation Data...")
val_data = val_datagen.flow_from_directory(
    val_dir, target_size=(IMG_HEIGHT, IMG_WIDTH), batch_size=BATCH_SIZE,
    class_mode='categorical', shuffle=False)

NUM_CLASSES = train_data.num_classes

# Common Callbacks Setup
early_stopping = EarlyStopping(monitor='val_loss', patience=4,
    restore_best_weights=True)
```

Loading Training Data...

Found 11793 images belonging to 4 classes.

Loading Validation Data...

Found 1123 images belonging to 4 classes.

2 MODEL 1 - MOBILENETV2

```
[12]: print("\n--- Building and Training MobileNetV2 ---")
```

--- Building and Training MobileNetV2 ---

2.0.1 Load the Base Model (pre-trained on ImageNet, excluding the final classification layers)

```
[13]: base_mobilenet = MobileNetV2(weights='imagenet', include_top=False,
    input_shape=(IMG_HEIGHT, IMG_WIDTH, 3))
```

/tmp/ipykernel_1308/2446740391.py:1: UserWarning: `input\_shape` is undefined or non-square, or `rows` is not in [96, 128, 160, 192, 224]. Weights for input shape (224, 224) will be loaded as the default.

```
base_mobilenet = MobileNetV2(weights='imagenet', include_top=False,
input_shape=(IMG_HEIGHT, IMG_WIDTH, 3))
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_224_no_top.h5

9406464/9406464

2s

0us/step

2.0.2 Freeze the base layers so we don't destroy the pre-trained weights

```
[14]: base_mobilenet.trainable = False
```

2.0.3 Adding custom Banana Classification Head

```
[15]: x = base_mobilenet.output
x = GlobalAveragePooling2D()(x) # Averages the 2D feature maps into a 1D vector
x = Dense(128, activation='relu')(x)
x = Dropout(0.4)(x)
predictions_mb = Dense(NUM_CLASSES, activation='softmax')(x)

model_mobilenet = Model(inputs=base_mobilenet.input, outputs=predictions_mb)
```

2.0.4 Compiling and Training

```
[17]: model_mobilenet.compile(optimizer='adam', loss='categorical_crossentropy',
    ↪metrics=['accuracy'])

checkpoint_mb = ModelCheckpoint('mobilenetv2_model.keras', save_best_only=True)

history_mb = model_mobilenet.fit(
    train_data,
    epochs=10, # Transfer learning requires fewer epochs to converge
    validation_data=val_data,
    callbacks=[early_stopping, checkpoint_mb]
)
```

```
Epoch 1/10
738/738          532s 696ms/step -
accuracy: 0.8899 - loss: 0.3129 - val_accuracy: 0.9412 - val_loss: 0.1690
Epoch 2/10
738/738          472s 639ms/step -
accuracy: 0.9287 - loss: 0.1986 - val_accuracy: 0.9412 - val_loss: 0.1608
Epoch 3/10
738/738          467s 633ms/step -
accuracy: 0.9427 - loss: 0.1627 - val_accuracy: 0.9671 - val_loss: 0.1073
Epoch 4/10
738/738          476s 646ms/step -
accuracy: 0.9483 - loss: 0.1442 - val_accuracy: 0.9608 - val_loss: 0.1135
Epoch 5/10
738/738          458s 620ms/step -
accuracy: 0.9517 - loss: 0.1323 - val_accuracy: 0.9733 - val_loss: 0.0846
Epoch 6/10
738/738          464s 629ms/step -
accuracy: 0.9545 - loss: 0.1257 - val_accuracy: 0.9706 - val_loss: 0.0939
Epoch 7/10
```

```

738/738          459s 622ms/step -
accuracy: 0.9538 - loss: 0.1238 - val_accuracy: 0.9724 - val_loss: 0.0853
Epoch 8/10
738/738          454s 615ms/step -
accuracy: 0.9601 - loss: 0.1153 - val_accuracy: 0.9599 - val_loss: 0.1172
Epoch 9/10
738/738          459s 622ms/step -
accuracy: 0.9596 - loss: 0.1084 - val_accuracy: 0.9724 - val_loss: 0.0932

```

3 MODEL 2 - EFFICIENTNETB0

```
[7]: print("\n--- Building and Training EfficientNetB0 ---")
```

```
--- Building and Training EfficientNetB0 ---
```

3.0.1 Load EfficientNet Base

```
[8]: base_efficient = EfficientNetB0(weights='imagenet', include_top=False,
    ↪input_shape=(IMG_HEIGHT, IMG_WIDTH, 3))
base_efficient.trainable = False
```

```

Downloading data from https://storage.googleapis.com/keras-
applications/efficientnetb0_notop.h5
16705208/16705208          2s
0us/step

```

3.0.2 Adding custom head

```
[9]: y = base_efficient.output
y = GlobalAveragePooling2D()(y)
y = Dense(128, activation='relu')(y)
y = Dropout(0.4)(y)
predictions_eff = Dense(NUM_CLASSES, activation='softmax')(y)

model_efficient = Model(inputs=base_efficient.input, outputs=predictions_eff)
```

3.0.3 Compiling and Training

```
[10]: model_efficient.compile(optimizer='adam', loss='categorical_crossentropy',
    ↪metrics=['accuracy'])

checkpoint_eff = ModelCheckpoint('efficientnetb0_model.keras',
    ↪save_best_only=True)

history_eff = model_efficient.fit(
    train_data,
```

```

    epochs=5,
    validation_data=val_data,
    callbacks=[early_stopping, checkpoint_eff]
)

```

```

Epoch 1/5
738/738          529s 675ms/step -
accuracy: 0.3325 - loss: 1.3557 - val_accuracy: 0.3455 - val_loss: 1.3435
Epoch 2/5
738/738          477s 646ms/step -
accuracy: 0.3405 - loss: 1.3485 - val_accuracy: 0.3455 - val_loss: 1.3403
Epoch 3/5
738/738          481s 651ms/step -
accuracy: 0.3395 - loss: 1.3462 - val_accuracy: 0.3455 - val_loss: 1.3395
Epoch 4/5
738/738          483s 655ms/step -
accuracy: 0.3409 - loss: 1.3449 - val_accuracy: 0.3455 - val_loss: 1.3386
Epoch 5/5
738/738          492s 667ms/step -
accuracy: 0.3409 - loss: 1.3439 - val_accuracy: 0.3455 - val_loss: 1.3375

```

4 QUICK COMPARISON OF MODELS

```

[18]: print("\n--- Training Complete! ---")
      print("Evaluating final Validation Accuracy for both models:")

      # Evaluate MobileNetV2
      mb_loss, mb_acc = model_mobilenet.evaluate(val_data, verbose=0)
      print(f"MobileNetV2 Validation Accuracy: {mb_acc*100:.2f}%")

      # Evaluate EfficientNetB0
      eff_loss, eff_acc = model_efficient.evaluate(val_data, verbose=0)
      print(f"EfficientNetB0 Validation Accuracy: {eff_acc*100:.2f}%")

```

```

--- Training Complete! ---
Evaluating final Validation Accuracy for both models:
MobileNetV2 Validation Accuracy: 97.33%
EfficientNetB0 Validation Accuracy: 34.55%

```